beniFIT Build Spec for Agents

Oct 23, 2026 · @ila

This spec tells the coding agents exactly what to build for the beniFIT hackathon demo: a web app where a person signs up, answers six questions, connects their insurance, uploads their benefits guide and bills, and Beni shows what they're owed, checks their bills and answers questions in English or Spanish.

How to use this spec

Build in the order of the last section, "Build order." Each step has acceptance tests; don't start the next step until they pass.

Rules for agents

1. **Use only the stack and folder structure in this spec.** Don't add databases, auth providers or extra frameworks.
2. **Money amounts come from code, words come from Gemini.** Every dollar amount Beni shows is computed by the rules engine or read from a claim. Gemini reads PDFs, matches plans and writes messages, but never invents a number.
3. **Every finding has a source** (claim ID, guide page or bill) and it's shown on the card.
4. **Demo data only.** All people, plans, providers and phone numbers are fictional. Never call a real insurer API with real member data.
5. **Wording:** "you may qualify" and "may be eligible"; never "you will get."
6. **Two languages:** every user-facing string lives in the language files (en, es). No hard-coded text in components.
7. **The app must run offline** from cached results (see Demo mode) in case Wi-Fi or Gemini fails on stage.
8. **Mobile-first layout,** max width 480px, centered on desktop, so it looks like a phone app on the projector.
9. **Keep the Gemini API key on the server only.** It must never reach the browser.
10. When something in this spec is unclear, choose the simplest option that passes the acceptance tests and leave a TODO comment.

The three demo people

Person	Login	Coverage	Language	Demo files
Maria Alvarez, 38, Brooklyn	maria@demo.benifit	Employer plan, ClearPath PPO (Brightway)	English	maria_profile.json, maria_insurance_fhir.json, Brightway_2027_Benefits_Guide.pdf, Mercy_Grove_Bill_480.pdf,

				ClearPath_EOB_MRI.pdf, Mercy_Grove_Itemized_Bill.pdf
Ana Torres, 26, Bronx	ana@demo.benifit	NY Medicaid, HarborCare	Spanish	ana_profile.json, ana_insurance_fhir.json, Bronx_Central_ER_Bill_1150.pdf
Joe Kowalski, 72, Queens	joe@demo.benifit	Medicare Advantage, SilverPath	English	joe_profile.json, joe_insurance_fhir.json, Main_St_Pharmacy_Insulin_Receipt.pdf

Demo password for all three: **beni2026**. New people can also sign up and connect to any of the three demo insurers.

Product and business

Product: beniFIT is a personal benefits assistant. Its helper, Beni, connects to a person's health insurance, reads their benefits guide and bills, and tells them in plain words about money and help they're missing: billing errors, unclaimed benefits, expiring money, and food and health programs they may qualify for.

Tagline: Know what you're owed, before, during and after.

Closing line for the pitch: Beni makes sure no one goes hungry or overpays for care because of fine print.

The problem

Fact	Source
Older adults leave about $58 billion a year unclaimed in SNAP, SSI and Medicare Savings Programs	NCOA
Only 38% of eligible older adults use SNAP, about 9.1 million missing out	NCOA
More than 4 in 10 people eligible for WIC don't get it	CBPP
Americans owe at least $220 billion in medical debt	KFF

Who it's for

User	Need	Example
Working people on employer plans	Find buried perks, catch bill errors, use money before it expires	Maria
Families on Medicaid	Food, transportation and pregnancy benefits, protection from wrong bills, help in their own language	Ana
Older adults on Medicare	Help paying premiums and drugs, food benefits, plan choices during Open Enrollment	Joe

What makes it different

- **Proactive:** lives in your planner and speaks up before money is lost; other tools only act after a bill or a denial
- **Everything together:** insurance claims, employer benefits, government food and money programs, and bills
- **On your side:** independent from insurers and employers
- **Any language:** starts with English and Spanish
- **New York first:** new Medicaid nutrition and doula benefits, a strong hospital financial assistance law, no asset test for Medicare Savings Programs

Competitors

Who	What they do	Gap
Sheer Health	Connects to insurance, explains bills, handles disputes	Only after the bill; no employer perks or food programs
Nayya	AI benefits guide sold to employers	Only if your employer buys it; no bill checks
AI appeal letter tools	Draft appeals for denied claims	One task, after the fact

Business model

Revenue	How

Free for families	Core findings, alerts, bill checks and chat
Partners pay per member	Clinics, community health organizations, nonprofits, unions and employers offer beniFIT to their people
Success fee (optional)	Small share of money recovered when beniFIT handles a dispute for you
Premium	Hands-on help with appeals and applications

Go to market

1. Pilot with 1 or 2 New York community health organizations or clinics that serve Medicaid and Medicare members
2. Measure dollars found per person and benefits used for the first time
3. Expand to employers and unions, then other states

Success metrics

- Dollars found and recovered per member
- Share of Beni's alerts acted on
- Billing errors caught before payment
- People enrolled in a food or money program for the first time

What this means for the build

The demo must prove three things in 3 minutes: Beni **catches a real bill error**, Beni **finds money and food help a person didn't know about**, and Beni **works for very different people, in their language**. Every screen in this spec serves one of those three.

Tech stack and repo structure

One Next.js app does both the front end and the backend, so there's one thing to run and deploy.

Part	Choice	Why
Framework	Next.js (App Router) + TypeScript	Pages and API routes in one project
Styling	Tailwind CSS	Fast, consistent cards

AI	Google Gen AI SDK for JavaScript (@google/genai), model name in GEMINI_MODEL (use a current Gemini Flash model)	Reads PDFs, returns JSON
Storage	JSON files on the server in /storage (users, profiles, findings, chats)	No database setup
Sessions	Signed HTTP-only cookie with the user ID (iron-session or a signed JWT)	Simple and secure enough for a demo
Passwords	bcryptjs hash	Never store plain passwords
PDF upload	Next.js route handler with FormData, files saved to /storage/uploads/<userId>/	
Validation	zod for every API request and every Gemini JSON response	Catches bad AI output
Hosting	Local for the demo; Cloud Run if time allows	

Environment variables (.env.local)

```
GEMINI_API_KEY=...
GEMINI_MODEL=gemini-flash-latest   # replace with the exact current Flash model id
SESSION_SECRET=at-least-32-random-characters
DEMO_MODE=true                     # serves cached results when Gemini fails
```

Repo structure

```
benifit/
  app/
    page.tsx                    # 1 Landing
    signup/page.tsx             # 2 Sign up
    login/page.tsx              # 3 Log in
    onboarding/language/page.tsx  # 4 Language
    onboarding/about/page.tsx     # 5 About you
    onboarding/connect/page.tsx   # 6 Connect insurance
    onboarding/consent/page.tsx   # 6b Fake insurer consent screen
    onboarding/documents/page.tsx # 7 Upload documents
    onboarding/analyzing/page.tsx # 8 Analyzing
```

```
    home/page.tsx              # 9 Beni found
    planner/page.tsx           # 10 Planner
    bills/page.tsx             # 11 Bill check
    chat/page.tsx              # 12 Ask Beni
    settings/page.tsx          # 13 Settings
    api/...                    # see API section
  components/  BeniAvatar.tsx  BeniCard.tsx  TotalFound.tsx  BottomNav.tsx
LanguageToggle.tsx  ProgressDots.tsx
  lib/
    session.ts  storage.ts  fhir.ts  rules.ts  gemini.ts  prompts.ts  i18n.ts  schemas.ts
  locales/  en.json  es.json
  demo-data/
    insurers.json              # the 3 demo insurers and which FHIR file each returns
    maria/  ana/  joe/         # the demo files listed above
    cache/  maria.json  ana.json  joe.json   # saved analyze, planner and bill-check results
  storage/                     # created at runtime, git-ignored
```

User journey

A new person goes through onboarding once (screens 2 to 8), then lands on the home screen. A returning person logs in and goes straight home.

1. **Landing** → "Get started" (sign up) or "Log in"
2. **Sign up:** name, email, password, agree to terms
3. **Language:** English or Español
4. **About you:** 6 optional questions (ZIP, language confirmed, household, income, pregnancy, Medicare premium)
5. **Connect insurance:** pick insurer → fake insurer login and consent screen → "Connected!"
6. **Upload documents (optional):** benefits guide and any bills
7. **Analyzing:** Beni reads everything (10 to 20 seconds, with progress messages)
8. **Home, "Beni found $X":** total plus findings cards
9. From the bottom nav: **Planner**, **Bills**, **Ask Beni**, **Settings**

Routing rules

Situation	Goes to
Not logged in, opens any page except /, /signup, /login	/login
Logged in, onboarding not finished	The next unfinished onboarding step

Logged in, onboarding finished, opens /, /signup or /login	/home
Insurance connected but analysis never run	/onboarding/analyzing

Store onboarding progress on the user as onboardingStep: language → about → connect → documents → analyzing → done.

Bottom navigation (after onboarding)

Four tabs with icons: **Home** (Beni found), **Planner**, **Bills**, **Ask Beni**. Settings is the gear icon at the top right. A language toggle (EN | ES) sits in the header on every screen.

Screens 1 to 5: getting started

1. Landing (/)

- Beni avatar (happy), app name **beniFIT**, headline: "Find what you're owed." Subline: "Beni reads your insurance, benefits and bills, and tells you about money and help you're missing."
- Three small proof points: "Catches billing mistakes" · "Finds hidden benefits" · "Alerts you before money expires"
- Buttons: **Get started** → /signup, **Log in** → /login
- Footer: "Demo app. All data is fictional."
- **Done when:** both buttons navigate correctly; text switches with the language toggle.

2. Sign up (/signup)

Field	Type	Validation
First name	text	required, 1 to 40 characters
Email	email	required, valid email, not already used
Password	password	required, at least 8 characters
I agree to the terms and privacy notice	checkbox	required

- Under the form: "Your data stays private. We never sell it. You can delete it any time."

- On submit: POST /api/auth/signup → create user with bcrypt hash, onboardingStep: "language", set session cookie → go to /onboarding/language
- Errors shown under the field: "This email is already registered. Log in instead?"
- **Done when:** a new user is saved in /storage/users.json with a hashed password, the cookie is set, and a refresh keeps them logged in.

3. Log in (/login)

- Email, password, **Log in** button, link to sign up
- Box below: "Try a demo: Maria · Ana · Joe" — three buttons that log in as the demo accounts in one click
- On success: go to the next onboarding step, or /home if done
- Wrong password: "Email or password is incorrect." (don't say which)
- **Done when:** demo buttons log in instantly and land on /home with findings already loaded.

4. Language (/onboarding/language)

- Beni: "Hi {firstName}! I'm Beni. Which language do you prefer?"
- Two large buttons: **English** · **Español** (room in the layout for more later)
- Saves profile.language and switches the whole app immediately
- Progress dots at the top: step 1 of 5
- **Done when:** choosing Español switches every screen to Spanish.

5. About you (/onboarding/about)

Beni: "A few quick questions help me find programs you may qualify for. All optional. I never share your answers." One question per card, with **Skip** and **Next**. Each card has a small "Why I ask" line.

#	Question	Input	Saved as	Why I ask
1	What's your ZIP code?	5 digits	zip	Programs and clinics depend on where you live
2	What's your date of birth?	date	birthDate	Some programs are for people 65+ or for kids
3	Who lives with you?	Stepper: adults (1 to 8), children (0	household: {adults, children: [{age}]}	Food programs depend on household size

		to 8); for each child, age		
4	About how much does your household earn each month, before taxes?	Number with $ prefix, or "Prefer not to say"	monthlyIncome (number or null)	Many programs have income limits
5	Is anyone in your home pregnant or did they have a baby in the last year?	Yes pregnant / Yes had a baby / No; if pregnant, due date	pregnancy: {status, dueDate}	Pregnancy unlocks extra food and care benefits
6	Do you pay your Medicare premium yourself? (only if age is 65+)	Yes / No / Not sure	paysMedicarePremium	You may qualify for help paying it

- On finish: POST /api/profile → onboardingStep: "connect" → /onboarding/connect
- **Done when:** answers are saved, skipped questions are saved as null, and question 6 only shows for people 65 or older.

Screens 6 to 8: connecting and reading

6. Connect insurance (/onboarding/connect)

Beni: "Let's connect your health insurance. I'll read your plan and your claims. You can disconnect any time."

- Search box + list of insurers from demo-data/insurers.json:

Insurer (fictional)	Type	Returns FHIR file	Needs guide upload?
ClearPath Health (employer plans)	Employer / commercial	maria_insurance_fhir.json	Yes, ask for employer benefits guide
HarborCare Medicaid	NY Medicaid managed care	ana_insurance_fhir.json	No

SilverPath Advantage	Medicare Advantage	joe_insurance_fhir.json	No

- Small text under the list: "Medicare, Medicaid and marketplace plans must let you connect apps like beniFIT by law."
- Option at the bottom: "My insurer isn't listed" → skip to documents (upload only)
- Tap an insurer → **6b. Consent screen** (/onboarding/consent?insurer=...), styled as the insurer's page, with a gray banner "Demo: simulated insurer login":
    - Member ID and password fields (pre-filled for demo)
    - "beniFIT is asking to see: your coverage, your claims and explanations of benefits." Checkboxes for each, all checked
    - Buttons **Allow** / **Cancel**
- Allow → POST /api/connect → server loads that insurer's FHIR file, parses it with lib/fhir.ts, saves Coverage and Claims to the user → back to /onboarding/connect showing "✅ Connected to {insurer}. Found {n} claims."
- **Next** → /onboarding/documents
- **Done when:** each insurer loads its file, the claim count is right (Maria 4, Ana 3, Joe 3), and Cancel returns without connecting.

7. Upload documents (/onboarding/documents)

Beni: "Got any papers? I can read your benefits guide and any bills." Two upload areas:

Area	Accepts	Type saved	Shown when
Benefits guide	PDF, up to 20 MB	guide	Always; marked "Recommended" for employer plans
Bills, receipts, EOBs	PDF, JPG, PNG, up to 10 MB each, several files	bill	Always

- Each file shows name, size and a remove (x) button
- Demo shortcut link: "Use sample documents" loads the person's demo PDFs
- **Skip** or **Next** → POST /api/documents (multipart) → onboardingStep: "analyzing" → /onboarding/analyzing
- **Done when:** files are saved under /storage/uploads/<userId>/ with type and original name.

8. Analyzing (/onboarding/analyzing)

- Beni avatar animating (thinking), progress bar, rotating messages every 2 seconds:

    - "Reading your plan…"
    - "Checking {n} claims…"
    - "Reading your benefits guide, page by page…"
    - "Looking for food and money programs near {zip}…"
    - "Checking your bills…"
- Calls POST /api/analyze once on load. On success → onboardingStep: "done" → /home with a confetti burst and Beni celebrating
- If it takes longer than 25 seconds or fails, and DEMO_MODE=true: load demo-data/cache/<person>.json silently
- If it fails without a cache: "Beni had trouble reading. Try again" button
- **Done when:** each demo person reaches /home with the right total (Maria $1,265, Ana $3,100, Joe $4,200).

Screens 9 to 13: the app

9. Home: Beni found (/home)

- Header: "Hi {firstName}!" + language toggle + settings gear
- **Total card:** Beni celebrating, big number counting up from $0: "Beni found **$1,265** for you this year." Small: "Includes estimates. Tap any card for details."
- **Filter chips:** All · Bills · Money waiting · Expiring · Food · Free help
- **Findings list,** sorted: bill errors first, then items with the nearest deadline, then by amount
- **BeniCard** (used everywhere):
    - Colored icon by type (bill_error red, expiring orange, money_waiting green, food yellow, hidden_benefit blue, coverage purple)
    - Title (one line), message (one or two sentences, in Beni's voice), amount ("$300" or "Free" or "about $1,200 a year")
    - Deadline badge if any: "⏰ Dec 31"
    - Source line in small gray: "From: Claim EOB-0004" or "From: Benefits guide p.5"
    - One action button: Call (opens tel:), Open app (opens link), Claim it / Apply (opens link or shows steps), Draft message (opens bill check result)
    - Secondary: "Done" checkmark (moves card to a Done section and adds to "You got $X") and "Not for me" (hides it)
- **Done when:** each demo person shows the expected findings and total; filters work; Done and Not for me persist after refresh.

10. Planner (/planner)

- Week view (list by day, today first). Each plan item shows its text; if Beni matched a finding, a BeniCard appears under it, slightly indented, with a Beni mini-avatar
- Add box at the bottom: "What's coming up?" + date picker → POST /api/planner adds the item and returns its match (or no match) within a few seconds
- Demo people start with the planner items from their profile file

- **Done when:** typing "Book Uber to my MRI" for Maria shows the free rides card; "Groceries, money is tight" for Ana shows WIC in Spanish.

11. Bill check (/bills)

- List of uploaded bills, each with status: "Not checked" · "Looks right ✅" · "Problem found ⚠️"
- **Upload a bill** button (PDF or photo)
- **Check** → POST /api/bill-check → result card:

Row	Example (Maria)
Bill says you owe	$480.00
Insurance says you owe	$180.00
Difference	**$300.00 too much**
What's wrong	The MRI was billed twice. Insurance denied the second line as a duplicate, and the hospital moved it to you.
Source	Mercy Grove bill MG-88213 vs. claim EOB-0004

- Below: **Drafted message** to the billing office in an editable text box, with **Copy**, **Email** (mailto:) and **Call billing** (tel:) buttons
- Also: "Don't pay this bill until it's fixed" tip, and "Ask for an itemized bill" if one isn't uploaded
- **Done when:** Maria finds $300, Ana finds $1,150 owed $0, Joe finds $85 on the insulin receipt.

12. Ask Beni chat (/chat)

- Chat bubbles: person on the right, Beni on the left with avatar
- Beni's first message: "Ask me anything about your coverage, your bills or help you may qualify for."
- Suggested chips: "What am I missing?" · "Is my bill right?" · "Can I get help with food?" · "¿Qué es WIC?"
- Each Beni answer ends with source tags (small gray pills: "Guide p.5", "Claim EOB-0004") and, if useful, one action button
- Beni answers in the language of the question

- If the answer isn't in the person's files: "I don't see that in your plan papers. You can ask {insurer} at {phone}."
- Keep the last 20 messages in /storage/chats/<userId>.json
- **Done when:** the 6 chat questions in the acceptance tests answer correctly with sources.

13. Settings (/settings)

- Profile: name, email, language
- About you: edit answers (reruns analysis on save)
- Connected insurer: name, connected date, **Disconnect** (removes coverage and claims)
- Documents: list, delete each
- **Delete all my data** (confirm dialog) → removes everything and logs out
- **Log out**
- Demo only: **Reset demo** (restores the demo person's original state)
- **Done when:** disconnect and delete actually remove the data from /storage.

Data model and FHIR mapping

Put these types in lib/schemas.ts as zod schemas and export the TypeScript types from them.

```
type Language = "en" | "es";
 type OnboardingStep = "language" | "about" | "connect" | "documents" | "analyzing" | "done";

 interface User {
   id: string;                // uuid
   firstName: string;
   email: string;             // lowercase, unique
   passwordHash: string;      // bcrypt
   createdAt: string;         // ISO date
   onboardingStep: OnboardingStep;
   demoPerson?: "maria" | "ana" | "joe";   // set for demo accounts
 }

 interface Profile {          // About you answers
   userId: string;
   language: Language;
   zip: string | null;
   birthDate: string | null;   // YYYY-MM-DD
   household: { adults: number; children: { age: number }[] } | null;
   monthlyIncome: number | null;
   pregnancy: { status: "pregnant" | "recent_birth" | "no"; dueDate?: string } | null;
   paysMedicarePremium: "yes" | "no" | "not_sure" | null;
 }

 interface Coverage {
   userId: string;
```

```
  insurerId: "clearpath" | "harborcare" | "silverpath";
  insurerName: string;
  planName: string;
  planType: "employer" | "medicaid" | "medicare_advantage";
  memberId: string;
  periodStart: string; periodEnd: string | null;
  costs: { label: string; amount?: number; percent?: number }[];  // deductible, OOP max...
  connectedAt: string;
}

interface Claim {
  id: string;                // EOB id, e.g. "EOB-0004"
  userId: string;
  type: "professional" | "institutional" | "pharmacy";
  provider: string;
  serviceDate: string;
  lines: {
    seq: number; code: string; codeSystem: "CPT" | "NDC"; description: string;
    billed: number; allowed: number; planPaid: number; memberOwes: number;
    denialReason?: string;
  }[];
  totalBilled: number; totalPlanPaid: number; totalMemberOwes: number;
  note?: string;
}

interface DocumentFile {
  id: string; userId: string;
  kind: "guide" | "bill";
  fileName: string; path: string; uploadedAt: string;
  billCheck?: BillCheckResult;
}

type FindingType = "bill_error" | "money_waiting" | "expiring" | "hidden_benefit" | "food" |
"coverage";

interface Finding {
  id: string;
  type: FindingType;
  title: string;             // in the person's language
  message: string;           // Beni's words, 1-2 sentences
  amount: number | null;     // dollars, from rules engine or claim
  amountLabel: string;       // "$300", "Free", "about $1,200 a year"
  isEstimate: boolean;
  deadline: string | null;
```

```
  source: string;              // "Claim EOB-0004", "Benefits guide p.5"
  action: { label: string; kind: "call" | "open_app" | "claim" | "draft" | "apply"; target: string };
  status: "new" | "done" | "dismissed";
  ruleId?: string;             // which rule produced it
}

 interface PlannerItem { id: string; userId: string; date: string; text: string; findingId: string | null;
cardMessage: string | null; }

 interface BillCheckResult {
  billAmount: number; insuranceSaysYouOwe: number; difference: number;
  problem: "duplicate_charge" | "denied_line_billed" | "medicaid_member_billed" |
"insulin_over_cap" | "more_than_eob" | "none";
  explanation: string; source: string; draftMessage: string; matchedClaimId: string | null;
}

 interface ChatMessage { role: "user" | "beni"; text: string; sources: string[]; action?:
Finding["action"]; at: string; }
```

Storage files

/storage/users.json (array of User) · /storage/profiles/<userId>.json · /storage/coverage/<userId>.json (Coverage + Claim[]) · /storage/findings/<userId>.json · /storage/planner/<userId>.json · /storage/documents/<userId>.json · /storage/chats/<userId>.json. Write with a temp file then rename, so a crash never corrupts a file.

FHIR mapping (lib/fhir.ts)

The demo FHIR files are Bundles. Map them like this:

Our field	FHIR path
Coverage.memberId	Coverage.subscriberId
Coverage.insurerName	Coverage.payor[0].display
Coverage.planName	Coverage.class where type = plan → name (or Coverage.type.text)
Coverage.costs	Coverage.costToBeneficiary[] → type.text + valueMoney.value or valueQuantity.value
Claim.id	ExplanationOfBenefit.id

Claim.type	ExplanationOfBenefit.type.coding[0].code
Claim.provider	ExplanationOfBenefit.provider.display
Claim.serviceDate	ExplanationOfBenefit.billablePeriod.start
line.code / description	item[].productOrService.coding[0].code / display
line.billed	item[].adjudication where category code = submitted
line.allowed	adjudication code = eligible
line.planPaid	adjudication code = benefit
line.memberOwes	adjudication code = memberliability
line.denialReason	adjudication code = denialreason → reason.text
Claim.note	processNote[0].text

The profile JSON files also hold the About you answers for the demo people (member.monthly_income, household, coverage.pays_part_b_premium_himself, member.language). The expected_beni_findings and demo_total_found fields are **test answers only**: never send them to Gemini or show them in the app.